`strict"` in them. What does it do and why might you want to include it?

13.2.6. Strict mode

When most of your JavaScript programs are written in CoffeeScript, it can be easy to forget JavaScript's problems. CoffeeScript protects you from some JavaScript follies, so the `"use strict"` pragma added in ECMAScript 5 doesn't appear in your programs often. But it's still important to understand how it relates to CoffeeScript.

Strict mode forbids some things in JavaScript that are dangerous; if you use them, you'll get an error. For example, with strict mode, using an undeclared global variable in JavaScript will cause a `ReferenceError`:

ECMAScript 5 example

```
failsStrictMode = function() {
  "use strict";
  undeclaredVariable = 3;
};

failsStrictMode()
// Reference Error
```

Of course, implicit variable declaration in CoffeeScript means that you won't see this error in CoffeeScript programs.

As you can see in the previous example, to enable strict mode you add the pragma `"use strict"` to some scope in your program. Any violations of strict mode inside that scope will cause an error.

The CoffeeScript compiler won't compile your program into something that breaks strict mode, but it won't complain if you break strict mode yourself. So when you *run* the following compiled program, the runtime will throw a syntax error:

```
"use strict"

failsStrict =
  duplicateKey: 1
  "duplicateKey": 2
```